

System and Network Security

Dr. Ashok Kumar Das

Center for Security, Theory and Algorithmic Research
International Institute of Information Technology, Hyderabad

E-mail: ashok.das@iiit.ac.in

URL: <http://www.iiit.ac.in/people/faculty/ashok-kumar-das>

Personal Home Page: <http://sites.google.com/view/iitkgpakdas/>

Message Authentication and Hash Functions

Message Authentication

- “Message authentication” is a process to verify the received message came from alleged source and has not been altered.
- Three procedures to produce an authenticator:
 - ▶ Message Encryption (Symmetric/public key)
 - ▶ Message Authentication Code (MAC)
 - ▶ Hash Function

Symmetric Encryption

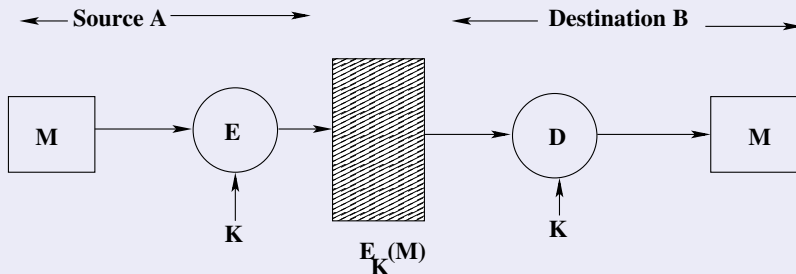


Figure: Symmetric encryption: confidentiality and authentication.

K : symmetric key shared between source A and destination B

Public-key Encryption

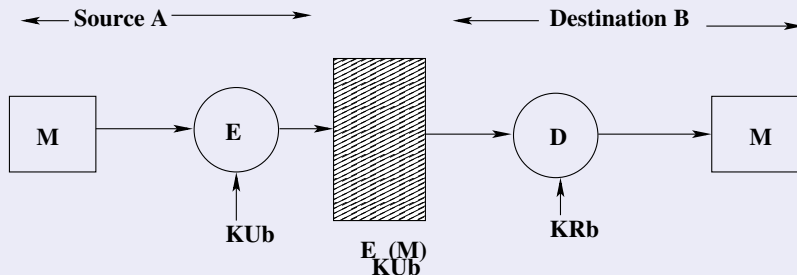


Figure: Public-key encryption: confidentiality.

KU_b : public key of destination B and KR_b : private key of B

Public-key Encryption

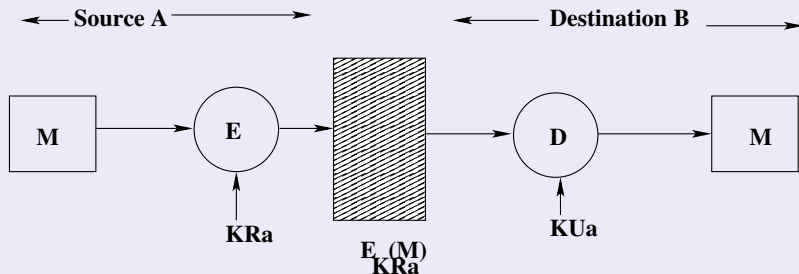


Figure: Public-key encryption: authentication and signature.

KU_a : public key of source A and KR_a : private key of A
 $E_{KR_a}(M)$: signature on message M created by the source A

Message Encryption

Public-key Encryption

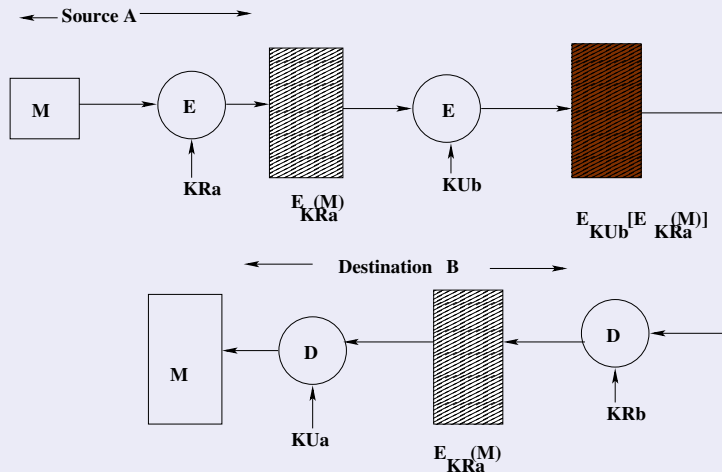


Figure: Public-key encryption: confidentiality, authentication and signature.

Message Authentication Code (MAC)

- A public function of the message and a secret key that produces a fixed-length value that serves as an authenticator.
- When user A has to send a message to user B , it calculates MAC as a function of the message and key K as $MAC = C_K(M)$, where M is the input message, C the MAC function, K the shared secret key and MAC the message authentication code.
Mathematically, $C_K : \{0, 1\}^* \rightarrow \{0, 1\}^n$.
- The receiver B is assured that the message has not altered. If an attacker alters the message but does not alter the MAC, then the receiver B 's calculation of the MAC will differ from the received MAC.
- Since the attacker does not know the secret key K , the attacker does not have the ability to alter the MAC.

Message Authentication Code (MAC)

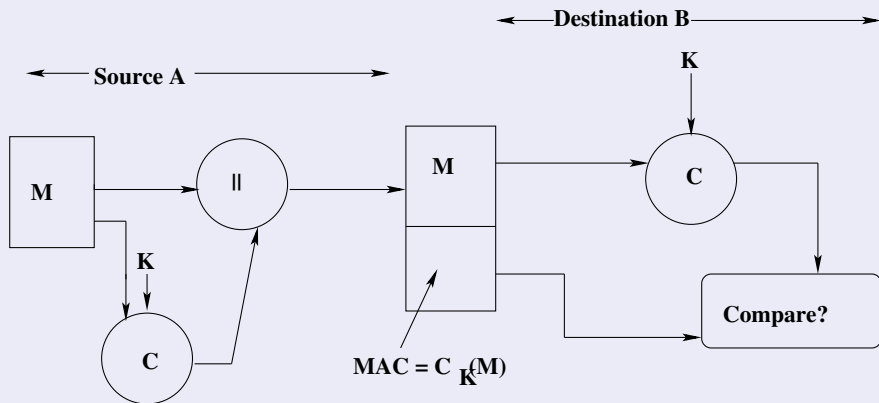


Figure: MAC: message authentication.

Message authentication

- $A \longrightarrow B : M || C_K(M)$
- Provides authentication
 - ▶ Only users A and B share the key K

Message authentication and confidentiality: authentication tied with plaintext

- $A \longrightarrow B : E_{K_2}[M || C_{K_1}(M)]$
- Provides authentication
 - ▶ Only users A and B share the key K_1
- Provides confidentiality
 - ▶ Only users A and B share the key K_2

Message authentication and confidentiality: authentication tied with ciphertext

- $A \rightarrow B : E_{K_2}[M] || C_{K_1}(E_{K_2}[M])$
- Provides authentication
 - ▶ Only users A and B share the key K_1
- Provides confidentiality
 - ▶ Only users A and B share the key K_2

Hash function

- A cryptographic hash function is an algorithm which accepts a variable length block of data as input and produces a fixed-size bit string, known as cryptographic hash value.
- Hash function can be applied to a large set of inputs which will produce outputs that are evenly distributed, and apparently random.
- Hash function provides data integrity.
- A change to any bit or bits in input data results, with high probability, in a change to the hash value.
- Mathematically, a one-way hash function $h : \{0, 1\}^* \rightarrow \{0, 1\}^l$ takes an arbitrary-length input $x \in \{0, 1\}^*$, and produces a fixed-length (say, l -bits) output $h(x) \in \{0, 1\}^l$, called the message digest or hash value.

Hash function

The hash function may be the fingerprint of a file, a message, or other data blocks, and has the following attributes.

- h can be applied to a data block of all sizes.
- For any given input x , the message digest $h(x)$ is easy to operate, enabling easy implementation in software and hardware.
- The output length of the message digest $h(x)$ is fixed.
- Deriving the input x from the given hash value $y = h(x)$ and the given hash function $h(\cdot)$ is computationally infeasible. This property is called the *one-way or pre-image resistance* property.
- For any given input x , finding any other input $y \neq x$ so that $h(y) = h(x)$ is computationally infeasible [*weak-collision resistant or second pre-image resistance* property].
- Finding a pair of inputs (x, y) , with $x \neq y$, so that $h(x) = h(y)$ is computationally infeasible [*strong-collision resistant or collision resistance* property].

Formal definition of a one-way hash function $h(\cdot)$

Definition (Collision-resistant one-way hash function)

A one-way collision-resistant hash function $h : \{0, 1\}^* \rightarrow \{0, 1\}^l$ is a deterministic algorithm that takes an input as an arbitrary length binary string $x \in \{0, 1\}^*$ and outputs a binary string $h(x) \in \{0, 1\}^l$ of fixed-length l . The formalization of an adversary $\mathcal{A}$'s advantage in finding collision is as follows.

$$Adv_{\mathcal{A}}^{HASH}(t) = Pr[(x, x') \leftarrow_R \mathcal{A} : x \neq x' \text{ and } h(x) = h(x')],$$

where $Pr[X]$ denotes the probability of an event X , and $(x, x') \leftarrow_R \mathcal{A}$ denotes the pair (x, x') is selected randomly by $\mathcal{A}$. In this case, the adversary $\mathcal{A}$ is allowed to be probabilistic and the probability in the advantage is computed over the random choices made by the adversary $\mathcal{A}$ with the execution time t . By an (ϵ, t) -adversary $\mathcal{A}$ attacking the collision resistance of $h(\cdot)$, we mean that the runtime of $\mathcal{A}$ is at most t and that $Adv_{(\mathcal{A})}^{HASH}(t) \leq \epsilon$.

References

- Palash Sarkar. A Simple and Generic Construction of Authenticated Encryption with Associated Data. ACM Transactions on Information and System Security, Vo. 13, No. 4, pp. 33, 2010.
- Douglas R. Stinson. Some Observations on the Theory of Cryptographic Hash Functions. Designs, Codes and Cryptography (Springer), Vol. 38, No. 2, pp.259-277, 2006.

Applications of a one-way hash function

- There are many applications of the hash functions, for examples, in the field of cryptology and information security, notably in digital signatures, message authentication codes (MACs), and other forms of authentication.
- Thus, a hash function becomes the basis of many cryptographic protocols.

Basic Uses of Hash Function $h(\cdot)$

Encrypt message plus hash code

- $A \longrightarrow B : E_K[M || h(M)]$
- Provides confidentiality
 - ▶ Only users A and B share the key K
- Provides authentication
 - ▶ $h(M)$ is cryptographically protected

Basic Uses of Hash Function $h(\cdot)$

Encrypt hash code- shared secret key

- $A \longrightarrow B : M || E_K[h(M)]$
- Provides authentication
 - ▶ $h(M)$ is cryptographically protected

Basic Uses of Hash Function $h(\cdot)$

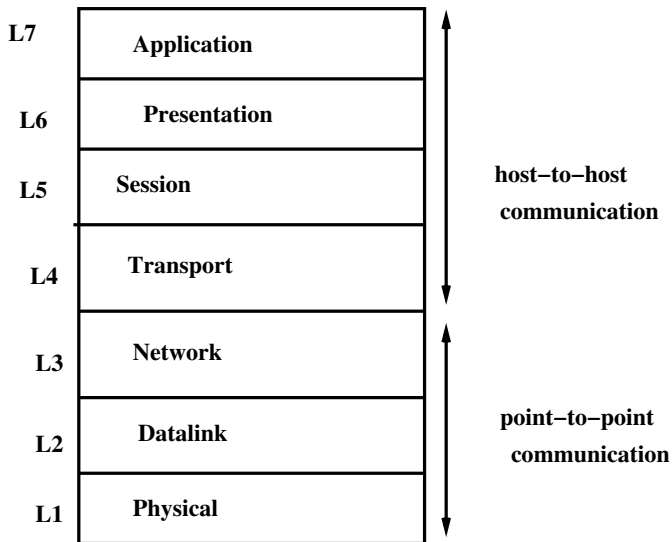
Encrypt hash code- sender's private key

- $A \longrightarrow B : M || E_{K_{R_a}}[h(M)]$
- Provides authentication and digital signature
 - ▶ $h(M)$ is cryptographically protected
 - ▶ Only A could create the signature $E_{K_{R_a}}[h(M)]$

Encrypting Communications Channels

- This is the classical Alice and Bob problem:
Alice wants to send Bob a secure message.
- What does she do?
- She encrypts the message.
- In theory, this encryption can take place at any layer in the OSI (Open Systems Interconnect) communication model.

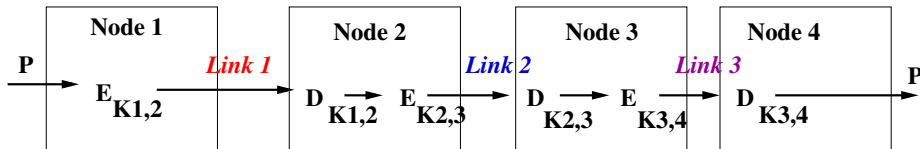
The OSI Reference Model



- In practice, it takes place either at the lowest layers (one and two) or at the higher layers.
- If it takes place at the lowest layers, it is called ***link-by-link encryption (LLE)***; everything going through a particular data link is encrypted.
- If it takes place at higher layers, it is called ***end-to-end encryption (EEE)***; the data are encrypted selectively and stay encrypted until they are decrypted by the intended final recipient.

- The easiest place to add encryption is at the physical layer.
- The interfaces to the physical layer are generally standardized, and it is easy to connect hardware encryption devices at this point.
- These devices encrypt all data passing through them, including data, routing information, and protocol information.
- They can be used on any type of digital communication link.
- On the other hand, any intelligent switching or storing nodes between the sender and the receiver need to decrypt the data stream before processing it.

Link-by-link encryption



P : plaintext message;

$K_{1,2}$: key shared between nodes 1 and 2;

$K_{2,3}$: key shared between nodes 2 and 3;

$K_{3,4}$: key shared between nodes 3 and 4;

$E_K(\cdot)$: encryption using the key K ;

$D_K(\cdot)$: decryption using the key K .

Advantages

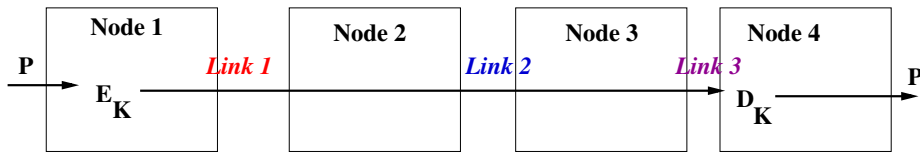
- Easier operation, since it can be made transparent to the user. That is, everything is encrypted before being sent over the link.
- Only one set of keys per link is required.
- Provides traffic-flow security, since any routing information is encrypted.

Disadvantages

- Data is exposed in the intermediate nodes.
- The biggest problem with encryption at the physical layer is that each physical link in the network needs to be encrypted: Leaving any link unencrypted reveals the security of the entire network. If the network is large, the cost may quickly become prohibitive for this kind of encryption.
- Additionally, every node in the network must be protected, since it processes unencrypted data. If all the network's users trust one another, and all nodes are in secure locations, this may be tolerable.

- This approach is to put encryption equipment between the network layer and the transport layer.
- The encryption device must understand the data according to the protocols up to layer three and encrypt only the transport data units, which are then recombined with the un-encrypted routing information and sent to lower layers for transmission.
- This approach avoids the encryption/decryption problem at the physical layer.
- By providing EEE, the data remains encrypted until it reaches its final destination.

End-to-end encryption



P : plaintext message;

K : key shared between nodes 1 and 4;

$E_K(\cdot)$: encryption using the key K ;

$D_K(\cdot)$: decryption using the key K .

End-to-end encryption

Advantages

- Higher secrecy level.

Disadvantages

- The primary problem with EEE is that the routing information for the data is not encrypted; a good cryptanalyst can learn much from who is talking to whom, at what times and for how long, without ever knowing the contents of those conversations.
- Key management is also more difficult since individual users must make sure they have common keys.
- Traffic analysis is possible, since routing information is not encrypted.

Combining the Two: Link-by-link encryption and End-to-end encryption

- Combining the two, while most expensive, is the most effective way of securing a network.
- Encryption of each physical link makes any analysis of the routing information impossible, while end-to-end encryption reduces the threat of unencrypted data at the various nodes in the network.
- Key management for the two schemes can be completely separate:
The network managers can take care of encryption at the physical level, while the individual users have responsibility for end-to-end encryption.

Comparing link-by-link encryption and end-to-end encryption

Link-by-link encryption	End-to-end encryption
Security within hosts	
1. Message exposed in sending host.	1. Message encrypted in sending host.
2. Message exposed in intermediate nodes.	2. Message remains encrypted in intermediate nodes.

Comparing link-by-link encryption and end-to-end encryption

Link-by-link encryption	End-to-end encryption
Role of user	
1. Applied by sending host. 2. Invisible to user. 3. Host maintains encryption. 4. One facility for all users. 5. Can be done in hardware. 6. All or no messages encrypted.	1. Applied by sending process. 2. User applies encryption. 3. User must find algorithm. 4. User selects encryption. 5. More easily done in software. 6. User chooses to encrypt or not, for each message.

Comparing link-by-link encryption and end-to-end encryption

Link-by-link encryption

Implementation concerns

1. Requires one key per host pair.
2. Requires encryption hardware or software at each host.
3. Provides node authentication.

End-to-end encryption

1. Requires one key per user pair.
2. Requires encryption hardware or software at each node.
3. Provides user authentication.